PAD: Separating the Stateless Brain from the Stateful Hub for Heterogeneous Long-Horizon Agent Fleets

Yanming Liu [1] **Xinyue Peng** [2]

Abstract

Long-horizon agent systems fail less from weak reasoning than from a runtime that loses track of what was asked, what has been done, and what counts as evidence that it was done. We present **PAD**, a hub that owns all durable task state, routes work by declared *capability* rather than by machine identity, and refuses to mark a task done unless the executor submits machine-checkable evidence for each acceptance criterion. A two-month deployment of six heterogeneous nodes—938 work tasks over 835 machine-hours, the longest running 42 hours—produced per-criterion evidence in 93% of completed tasks and a structured report in 42% of failed ones. It also showed that half of all failures are executor-environment faults rather than reasoning failures, which a hub can see and a single-loop harness cannot.

1. Introduction

A language model that can call tools is not yet a system that can be trusted with a week of work. The gap is not reasoning quality but *state management*. When a task spans several hours, several machines, and several failure modes, the questions that decide success are mundane: which subtask is currently running, on which host, under which credentials, and what evidence exists that step three actually happened rather than merely being reported as happened.

The agent-harness literature has largely answered this by enlarging the container. Harnesses keep a growing transcript, add planning and audit roles inside the same loop, or let an agent spawn recursive copies of itself with the same filesystem (Ma et al., 2026; Lumer et al., 2026; Zheng et al., 2026). This works well when the execution environment is one machine and the task fits in one context. It degrades predictably when the work is distributed: the transcript becomes a log of one host's view, and a self-assessment made inside that loop propagates into later decisions (Ma et al., 2026). A recent survey of persistent-state agents makes the same diagnosis from the other end—the literature concentrates on *accumulating* state and much less on governing, recovering, and relinquishing it (Ding et al., 2026).

This report describes a system built on the opposite premise. We separate two things that mainstream harnesses fuse:

- a **stateless brain**—a general chat model that can only ask "what is the situation" and "do this, and here is what would count as done";
- a **stateful hub**—a single process with a single database that owns every durable fact: who is alive, what is queued, who claimed what, what came back, and what evidence accompanied it.

The brain never holds a host name, never issues a shell command, and never sees an executor's log. The hub never reasons, never plans, and never retries with judgment. Everything that varies—models, CLIs, operating systems, credentials—lives on the executor side behind a declarative capability table.

Contributions.

1. **An architecture** (§3) that makes the model's interface to a fleet nine tools wide, and makes topology changes invisible to it.
2. **A dispatch contract** (§3.4) in which tasks are goals with acceptance criteria, and completion is gated on evidence per criterion rather than on an exit code.
3. **A deployment study** (§6) over 938 real work tasks and 835 machine-hours on six heterogeneous hosts, quantifying evidence coverage, failure-report quality, cancellation propagation, and where the design does *not* pay off.
4. **A failure taxonomy** (§7) from 131 failed tasks, with the practical lesson that most failures are executor-environment failures, not reasoning failures—which is

[1]Zhejiang University, Hangzhou, China [2]Southeast University, Nanjing, China. Correspondence to: Yanming Liu <oceann24@zju.edu.cn>, Xinyue Peng <xinyuepeng@seu.edu.cn>.

Technical report.

precisely what a hub can see and a single-loop harness cannot.

What this is not. PAD is not a new agent algorithm and does not improve any model. It is an argument about where state should live, with an implementation and evidence. We position it against harness work in §2 and are explicit in §8 about the three costs it introduces.

2. Related Work

Harness engineering. The framing that agent capability is increasingly *externalized* into memory, skills, protocols, and runtime scaffolding rather than into weights is now well established (Zhou et al., 2026). DeepSeek Harness takes this to its logical end for a single execution environment: everything, including the agent loop and the model adapter, is a replaceable plugin composed at boot (DeepSeek-AI, 2026; Cordis Authors, 2026). That design optimizes *extensibility within one host*. PAD optimizes *routing across many hosts*; the two are complementary, and in our deployment the per-node executor is exactly such a harness CLI, invoked headlessly.

Long-horizon state. LongHorizon-Harness reformulates long-horizon execution as task-state management, keeping state outside execution and updating it only with facts verified from the environment, via a manage–execute–audit loop (Ma et al., 2026). OneDayAgent argues that goal drift, state loss, and context overflow must be handled jointly by one harness (Zheng et al., 2026). We share the diagnosis and differ in placement: they keep the state inside the harness that also executes; we put it in a hub that does not execute at all, so that a node crash cannot take the ledger with it. Recursive Agent Harness studies recursion over full harnesses as a scaling strategy (Lumer et al., 2026), and self-referential agents take this further by letting the loop rewrite itself (Yin et al., 2024); PAD achieves a similar fan-out across *machines* without recursion, because the hub is the shared parent for every node.

Delegation and multi-agent coordination. SearchSwarm studies when a main agent should delegate and how much context to return (Lan et al., 2026); goal-aware collaboration work shows verification is the strongest predictor of success but tends to occur late (Shen et al., 2026). MAST catalogues failure modes of multi-agent LLM systems (Cemri et al., 2025). PAD sidesteps the hardest of these by not letting agents talk to each other: all coordination is mediated by the hub as task records, so there is no inter-agent conversation to drift.

Agent architectures. The practical distinction between a fixed workflow and an agent that decides its own steps (Anthropic, 2024) is exactly the line PAD draws when it dispatches goals rather than commands; our experience matches the guidance that deterministic pipelines are better built as workflows and reserved for their own capabilities. The classical decomposition into planning, memory, and action (Sumers et al., 2023; Wang et al., 2023b), and the line of work on interleaving reasoning with acting (Yao et al., 2023), verbal self-reflection (Shinn et al., 2023), skill libraries (Wang et al., 2023a), and tool learning (Schick et al., 2023), all assume a single agent loop that holds its own state. PAD deliberately removes that assumption and asks what remains when the loop is somewhere else.

Protocols and durable execution. MCP standardizes model-to-tool interfaces and is what our brain-side surface speaks (Model Context Protocol, 2025); its security literature motivates keeping the tool surface small and the authority narrow (Bhatt et al., 2025). Durable-execution systems (Temporal Technologies, 2026) and distributed task frameworks (Moritz et al., 2018) solve analogous problems for deterministic workflows. PAD borrows their vocabulary (claim, lease, heartbeat, idempotency key) but targets a setting where the unit of work is an LLM agent whose completion is not mechanically decidable—hence the evidence gate.

3. Architecture

3.1. Two doors, one state

PAD exposes exactly two surfaces, both keyed by a path secret:

- `/pad/<KEY>/mcp` — served over the Model Context Protocol (Model Context Protocol, 2025). Nine tools, all about tasks: read the situation, submit, inspect, cancel, fetch an artifact, fetch an input file. No shell, no filesystem, no machine identifier.
- `/pad/<KEY>/api/...` — a small REST surface for executors: register, heartbeat, claim, report, close.

Both write to one SQLite database. There is no message bus, no cache, no second source of truth. The persistence layer is deliberately boring: five tables (`nodes`, `tasks`, `reports`, `artifacts`, `task_inputs`) and append-only reporting. Idempotency is a client-supplied key on each report, so a node that loses a response can retry without double-counting.

3.2. Capability routing, not machine addressing

A task names a *capability*, never a host. An executor declares which capabilities it can serve; the hub matches work to declarations. A task asking for `scrape.batch` lands on whichever node has offered it, and the brain is never in-

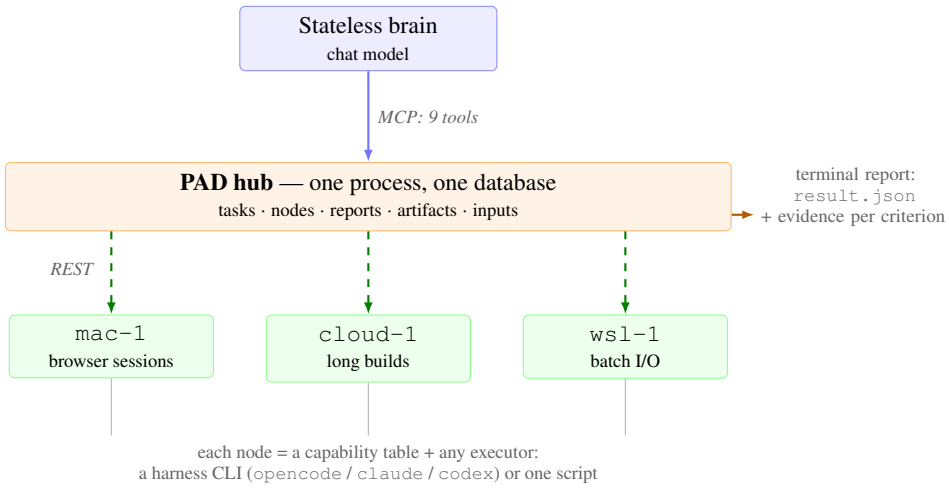


Figure 1. PAD separates the brain, the ledger, and the executors. The brain sees only nine task-management tools and never learns a host name or an operating system. The hub never executes anything and never reasons. An executor is anything that can claim work over REST and write a `result.json`: a plugin-composed harness CLI from one vendor, a different vendor's agent loop, or a deterministic script. Capabilities are matched by name, so the three nodes may be replaced or reordered without the brain changing.

formed which one answered. Three mechanical rules follow from that (§4 gives the reasoning):

1. **Topology is a deployment detail.** Machines were added, renamed, and removed during the measured period with zero changes on the brain side.

2. **Concurrency is an executor decision.** A capability may declare `concurrency: N`, which gives the node N slots; the node only claims work when it has a free slot.

3. **Conflict must be declared, not inferred.** If two machines doing the same thing would collide (shared credentials, shared output file, shared resource), exactly one may declare that capability. Read-only work may be declared by many under the same name, which is how the fleet scales horizontally. Renaming a conflicting capability to dodge the rule does not help—the hub would route both.

3.3. The shape of a dispatched task

A task carries a goal, a list of constraints, a list of acceptance criteria, an optional deadline, and optional input files. It carries no steps and no command. The executor is told: pursue the goal, stay inside the constraints, and if you must choose, choose a path that ends with evidence.

3.4. Evidence-gated completion

Every terminal report must answer the acceptance criteria one by one:

Field	Meaning
`status`	`done` or `failed`
`summary`	one line, for list views and heartbeats
`report`	prose: conclusion, what was achieved, per-subtask outcome, risks
`acceptance_results`	one entry per criterion: `passed` + `evidence`
`artifacts`	files with name, rows, schema, size, upload flag

The hub computes a coverage record on receipt and attaches it to the report (`missing` lists criteria that were not answered). It judges only *whether a criterion was addressed*, never *whether the answer is true*— truth is left to an independent reviewer node, which is a separate capability. This split matters: an in-loop self-audit (as in (Ma et al., 2026)) and a hub-side coverage check answer different questions and we use both.

Two secondary rules complete the contract. Data volume is compressed but answers are not: large raw output stays on the node and the hub keeps a pointer with row counts and schema, while fields the dispatcher named explicitly must be reproduced in full. And a failed task must still deliver (§4): partial results, the attempted paths, where it got stuck, and what evidence exists.

3.5. Executor-agnostic nodes

A node is a YAML file and a daemon; each capability maps to a shell command or to a thin adapter that renders a prompt template and invokes an agent CLI headlessly (Appendix B lists the four adapters we use). Because the adapter, not the hub, knows the vendor, machines from different vendors

coexist under identical capability semantics. This is the concrete mechanism behind the "heterogeneous" claim, and it is why a fleet can be assembled from hardware the operator already owns.

4. Design Principles

The architecture in §3 is a set of mechanisms. This section states the six principles that produced them, because the same mechanisms can be assembled for different reasons and the reasons decide what happens at the edges. Each principle is stated as a tension, the choice we made, and the failure it was chosen to prevent.

4.1. State must outlive the process that produced it

Every long-horizon system has two processes that die: the context window, which ends with the session, and the executor, which ends when the machine reboots or the CLI crashes. The conventional response is to enlarge the first—longer transcripts, summaries, retrieval over prior turns—which treats the context window as storage. It is not storage: no eviction policy, no schema, no query language, no sharing, no audit trail—and it is the component most likely to be truncated exactly when the task becomes long.

We place every durable fact at the layer with the longest mean time to failure: one process, one SQLite file, append-only. Brain and executor are both stateless clients that may be discarded and rebuilt from the ledger. This is a survivability argument rather than a database one, and it yields what context-based designs cannot have: the ledger is data, so "what ran since Tuesday" is a query, not a reconstruction.

4.2. The interface must be narrower than the capability

A harness grows by adding tools, and each is locally reasonable but globally expensive: more to hold in mind, more attack surface, and more failure modes because tools interact. Nine is not a budget picked for elegance; it is the set that survives "does the brain need this, or does the executor?" Everything that operates on the world—shell, filesystem, browser, network, credentials—belongs to the executor, which already has a machine to act on. The brain gets only operations on *work*.

The narrowness pays twice. It is a security property—the tool surface is the authority surface, and nine tools can be reasoned about completely—and a generalization property: a chat model with no tool-specific training drives PAD on the first attempt, because there is nothing to learn beyond "ask what the situation is, then say what should happen".

4.3. Route on what, never on where

Naming a host in a task is the standard way to lose composability. Once a dispatcher writes `run it on machine-3`, it must know the fleet, the fleet cannot change without editing the dispatcher, and the audit trail records a machine rather than an intent.

Capability routing requires the hub to know *what* and never *where*. Machines were added, renamed, and removed with no change on the brain side, because the brain was never told about them. Two corollaries are easy to miss. Concurrency becomes local—a capability declares `concurrency: N` and the node claims only when it has a free slot—so the hub never models load. And conflicts must be declared: if two machines would collide over shared credentials or a shared output file, exactly one may declare the capability, and renaming it to dodge the rule does not help because the hub would route both.

4.4. The edge knows its environment better than the center

A dispatch that specifies steps claims the dispatcher knows the executor's environment. That claim is usually false: whether a CLI needs `--skip-git-repo-check`, whether a repository is clean, whether a cookie has expired, whether the disk has room—all visible only at the edge, and all of it can change between dispatch and execution.

We therefore dispatch goals, constraints, and criteria, never steps. The testable consequence: one capability label routes to three vendors' CLIs across six machines, and the dispatch text never mentions which, because it cannot know. The price is that an executor may choose a path the dispatcher would not have; the alternative is a dispatcher whose every step is a guess about an environment it does not occupy.

4.5. Evidence is an artifact, not an assertion

Completion as an exit code is adequate when the command and the goal are the same thing, and useless when the goal is "produce a verified summary of this week's domestic admissions announcements"—there is no exit code for that. The system that ran the command is also the least able to judge whether the goal was met, because it has an interest in being done.

PAD therefore treats completion as a reporting obligation. Every terminal report answers the acceptance criteria one at a time, each with the evidence that satisfies it, and the hub computes a coverage record that lists exactly which criteria went unanswered. The hub judges only *whether a criterion was addressed*, never *whether the answer is true*; truth is the job of a separate reviewer capability. Keeping that boundary explicit is what makes the number meaningful: 93% of

completed tasks carry per-criterion evidence is a statement about reporting, not about correctness, and we state it that way throughout.

4.6. Failure is a first-class outcome

In most harnesses a failure is an exception: the loop breaks, the trace is printed, a human reads it. In long-horizon work this is the wrong shape, because failure is the common case in the tail. A task that ran for four hours and discovered that the goal cannot be met as stated has produced value—it has produced a finding—and a system that discards it as an error has discarded the finding.

Failed tasks in PAD must still deliver: partial results, the paths attempted, where it got stuck, and what evidence exists. This is what makes §7 possible at all. Half of the 132 failures are executor-environment faults, which is a fact about the fleet that is invisible to anyone who only counts successes.

4.7. What we deliberately did not build

Four things were refused on principle; Appendix A states each refusal and what it would have cost. The shortest summary: no planner, no second source of truth, no mid-task dialogue, no model in the routing path.

4.8. The brain slot is swappable

The word "agent" is used for two things that differ in who holds authority, and conflating them is the source of most of the confusion around personal assistants. A *personal agent* is given standing authority: it decides what to do, when, and with which resources, and its behavior is a function of its own judgment. A *work agent*—in our sense—has bounded authority: a principal states an outcome, and the agent executes inside a declared envelope. The first wants autonomy; the second wants accountability. Systems that pick one and call it done leave the other use case to be built separately.

PAD's position is that this is not a property of the architecture at all. It is a property of *which client occupies the brain slot*, and the hub should be indifferent. Three clients occupy that slot in the measured deployment, and the hub cannot tell them apart:

Client	Authority	Entry point
chat model	delegating: operator asks in prose, model picks capabilities and task count	MCP `/mcp`, `created_by=chatgpt`
human	direct: operator names a capability and a goal	`padctl submit`, `created_by=human`
scheduler	periodic: same pipeline, every day	timer, `created_by=daily-auto-timer`

All three write the same `tasks` row through the same `submit_task` function, get the same acceptance gating, and appear in the same digest. Table 1 splits the deployment by which client held the slot. Nothing downstream branches on `created_by`. This is the operational meaning of "stateless brain": the brain is not a component but a *role*, and a role that any client satisfying the protocol may hold.

Table 1. The same hub, three brain clients (work tasks only). Every column is a consequence of who held the slot, not of a different code path: the hub does not branch on the submitter field.

	chat model	human	timer
tasks	854	39	36
completed	680	36	19
failed	111	2	17
cancelled	61	1	0
evidence coverage	96%	47%	100%
median duration	14.6 min	0.7 min	341 min
machine-hours	632.9	6.9	194.4
median goal length	301 ch	131 ch	611 ch
with acceptance criteria	98%	79%	100%
capabilities touched	12	8	1

The consequences are worth stating because they are what makes the two modes coexist rather than conflict.

The split is not subtle, and it is legible in three directions at once. Work delegated to a chat model is *specified*: a median goal of 301 characters, 98% carrying acceptance criteria, spread over 12 capabilities. Work the operator submits by hand is *directed*: a median goal of 131 characters, 79% with criteria, a median duration under a minute, and only 6.9 machine-hours across the whole period. The scheduler is at the other extreme: one capability, the longest goals, and a 47% failure rate—because a fixed pipeline that runs unattended has no one to notice that its target moved.

The coverage column is the one to read carefully. Human submissions show 47%, which is not a defect: those are short direct commands that carry a goal and often no criteria, so there is nothing to report against. The scheduler shows 100% because it always writes criteria and always answers them. This is why we report coverage as a property of completions rather than of the system overall.

Switching modes costs nothing and requires no migration. In the measured deployment 10 of 46 active days contain tasks from both a chat model and a human. The operator typically works by hand when the task is well understood and the desired result is a single command, and hands the same capability to a chat model when the task needs research, decomposition, or a sequence of submissions. Both paths leave the same audit trail, so the record does not fragment when the mode changes.

Authority is expressed as policy, not as a different system. What separates a cautious work agent from an autonomous assistant is not a different sandbox or tool set; it is how much

of the envelope is fixed in advance. A `padctl` submission states capability, goal, and criteria in one shot with a human deciding each field. A chat model holds the same nine tools and therefore the same ceiling, but composes many submissions and chooses capabilities itself—which is why its goals run twice as long (Table 1): more delegation means more specification.

Granting more autonomy is a rate limit, not a redesign. A personal-assistant mode is PAD with a client that submits frequently, holds a long-lived session, and is trusted with broad capabilities. Nothing in the hub changes; what changes is which capabilities the operator declares, which is a YAML edit. We consider this the correct place for the decision, because it keeps the authority grant explicit and revocable instead of embedding it in the agent's prompt.

But autonomy has a real cost the data shows. The same mechanism that makes mode switching free makes over-delegation easy: a chat model with a wide capability list will happily submit a goal that no capability can satisfy, and the hub will record a failure. Half the failures in §7 are exactly this shape. A system that gave the model more authority over routing would not have produced a failure taxonomy at all, because it would have had no single place to record what went wrong.

4.9. Coordination without a coordinator

The natural objection to a hub is that it is centralized, and centralized coordination is usually a bottleneck. PAD is centralized in state and decentralized in execution, and the distinction is what allows six machines to be coordinated without a scheduler process.

The hub holds no queue worker and runs no task itself. It answers two questions per poll: *is there work matching my declared capabilities*, and *what is the state of the work I hold*. Every node polls on its own schedule and claims only when it has a free slot, so throughput is the sum of node-local decisions rather than the output of a global scheduler. There is no assignment step anywhere in the system: a task is claimed, not dispatched, and the hub's routing table is a set of declarations rather than a set of decisions.

Four properties follow, and each replaces a piece of machinery a conventional orchestrator would need.

No scheduler, no consensus. Because work is claimed from a table keyed by capability, the hub needs no global view of load and no protocol to agree on an assignment. Adding a machine is a registration; removing one is a timeout.

Backpressure is automatic. A capability declaring `concurrency: N` is a node that will not claim its $(N{+}1)$-th task. The queue grows in the database, which is where it should be visible, rather than in a broker's memory.

Failure is localized. A node that dies mid-task leaves a task in `running` and a lease that expires; because reports are append-only and idempotent, the next holder of the capability can resume from the ledger rather than from a process's memory.

The coordination record is the audit record. Every claim, report, and artifact is a row. This is why the failure taxonomy in §7 required no instrumentation effort: the data a coordinator would have emitted for its own purposes is the same data an auditor needs.

The price is real and we state it in §8: one hub is a single point of coordination, and a restart drops process supervision for in-flight tasks. For a single operator with a handful of machines, that trade is clearly correct; for a multi-tenant fleet it would not be, and PAD makes no claim there.

5. Operational Model

5.1. What the brain can do

The nine tools form a closed vocabulary: read state; submit; inspect one task; read an artifact by paged offset; read an input file; cancel. A general chat model can drive this without tool-specific training, and—importantly—without being told anything about the fleet. In practice a session begins with one situation read and proceeds with submissions whose goals reference capabilities the model saw there.

This narrowness is a security property as much as a usability one. The tool surface is the authority surface; a fetch-by-URL path that accepts arbitrary remote content is the only place where the hub touches the network on behalf of an external party, and it is written to refuse private and loopback resolution, re-validate every redirect hop, and abort on size limits.

5.2. Reporting discipline

We require progress reports to be one line and terminal reports to be prose, and we separate the two deliberately. Progress exists so the brain can maintain a world model cheaply; terminal reports exist so a human (or a reviewer node) can audit. Logs and stack traces belong to neither: they stay on the node as artifacts. Compressing volume while keeping pointers is what makes a 42-hour task inspectable without flooding a context window.

5.3. Adding work

A new capability is a YAML entry, not a code change. In the measured deployment this produced a long tail of narrow capabilities (a scheduled content pipeline with ordered dependent steps, a scraping batch job, a per-machine self-check) beside a few broad ones (`*.generic` for judgment

Table 2. The measured fleet. "Account state" means browser or platform sessions that only one machine may hold; such capabilities are single-owner by the conflict rule of §3.

Node	Caps.	Hours	Role
laptop (macOS)	7	90.2	publishing (browser)
cloud x86	2	239.4	production host, long builds
linux aibook	3	266.9	code & data, two executors
WSL host A	4	145.0	batch I/O, overseas
WSL host B	3	85.9	batch I/O
domestic low-mem	2	7.4	domestic reach, 1.4 GB
total	20	834.8	6 machines, 3 vendor CLIs

Table 3. Duration and outcome by bucket (work tasks only, $n = 938$). Evidence coverage is the fraction of completed tasks whose terminal report answers every acceptance criterion.

Duration	n	done	failed	canc.	evidence
< 10 min	388	295	66	25	89%
10 min–1 h	375	319	31	25	97%
1–6 h	150	112	27	11	96%
> 6 h	25	16	7	2	100%
all	938	742	131	63	93%

work). The tail is a feature: narrow capabilities carry operational knowledge the dispatcher should not have to restate every time.

We note one counter-pattern we hit and document for others: naming a capability after a workflow invites the brain to apply that workflow to goals that merely resemble it. A label like `mac.daily.all` implicitly means "run today's fixed pipeline", so a request to prepare tomorrow's output gets today's date baked in. Our rule is that fixed-workflow capabilities are used only when the intent is literally "as usual"; anything novel routes to a generic capability where planning stays with the executor.

6. Deployment Study

6.1. Setting

We instrumented a live deployment over roughly two months. It is not a benchmark: it is the operator's actual work, including the failures. Table 2 describes the fleet at measurement time.

The corpus is 975 tasks, of which we exclude 37 self-check tasks (`*.ping.self`) from the behavioral analysis, leaving **938 work tasks**. Two tasks were still running at measurement. Machine-hours are *occupied* hours, computed as created-to-terminal wall time summed per node; they include queueing, because queueing is what the operator experiences.

6.2. Scale and duration

Table 3 is the headline. Median work duration is 14.5 minutes, but the distribution has a long tail: 18.7% of tasks exceed one hour, 25 exceed six hours, and the maximum is 42.1 hours. Evidence coverage *rises* with duration—89% in the shortest bucket against 97–100% in the rest—which is the opposite of what one would expect if long tasks were where the contract breaks down. Our reading is that long tasks are exactly where executors take the reporting requirement seriously, because a long task that returns without evidence has obviously failed its purpose, whereas a two-minute task can slide by on a one-line "done".

6.3. Where the time goes

Separating queueing from execution, median wait from submission to first report is 13 seconds (p90 112 s, p99 5 987 s) while median execution is 813 s (p90 7 134 s, p99 24 711 s). The p99 queue figure reflects deliberate backlog, not contention: when all compatible nodes are busy, work waits, and the hub reports that honestly instead of failing fast.

Load across nodes is uneven (7–267 h, Table 2), a property of the capability map rather than of a balancer. The low-memory domestic box contributed 0.9% of machine-hours because its declared work is narrow; the value of a heterogeneous fleet is *reach*, not equal utilization.

6.4. Evidence and the failure path

Of 742 completed work tasks, **693 (93%)** carry per-criterion acceptance evidence. Of 131 failed tasks, **55 (42%)** still carry structured acceptance results and 55 attach artifacts; the average failed report is 862 characters of prose describing what was attempted, what was delivered, and where it broke. This is the property the design was built for: a failure is an artifact, not an absence.

Across the 933 tasks carrying acceptance criteria, 734 (79%) produced any acceptance results, and 483 of those left no criterion unanswered. We report both because they measure different things: whether an executor reported against the criteria at all, and whether the report left a gap. We report the low figure honestly: older tasks predate the mandatory-evidence rule, and the check distinguishes "answered nothing" from "answered wrongly". The temporal trend is what matters—the fraction rose monotonically after the rule became mandatory, and in the current period the completion path virtually always answers every criterion.

6.5. Cancellation propagation

63 tasks were cancelled, 61 of them after emitting at least one progress report. Median interval from the last progress report to the recorded cancellation is 8.8 minutes, which is

Table 4. Dominant cause of the 132 failed tasks, from a fixed first-match rule over the terminal report text (`tools/classify_failures.py`). Categories are mutually exclusive, so the column sums to 132.

Cause	n
Executor environment (CLI, PATH, crash)	64
Timeout / interruption	16
Goal not achievable as stated	16
Credentials or authorization	15
Unclassified	21

dominated by long-tail cases where the operator cancelled a task that had gone quiet; the maximum is 35 hours and reflects exactly the failure mode that motivated status polling. We report both rather than the median alone: the mechanism works, but coarse reporting can leave a long silence before anyone notices.

7. Failure Taxonomy

We classified the 132 failed tasks by the dominant cause visible in their terminal reports (Table 4).

This distribution is the strongest argument for the architecture. Nearly half of all failures (64 of 132) are *executor-environment* faults: a CLI not on the keep-alive PATH, a crash, a missing binary. A single-loop harness sees a broken tool call and can only retry; a hub sees a per-node, per-capability pattern and the operator fixes the environment once. That is the difference between debugging an agent and operating a fleet.

The unachievable-goal class holds the interesting cases—a licensing dead end, a search backend whose schema drifted, an authorization judgment later judged too broad—each of which produced a structured report a human then acted on. That is what makes "failed" a first-class outcome rather than an error.

8. Discussion

What this buys. Topology independence: 20 capabilities across 6 machines and 3 vendor CLIs, with the brain's interface unchanged. Evidence at scale: 93% of completions audited per criterion, rising rather than falling with task length. Operational visibility: the failure taxonomy of §7 is a query.

What it costs. Three costs, honestly.

First, describing work is the operator's burden. A goal with acceptance criteria takes longer to write than a command, and a vague criterion produces a vague answer. There is no way around this: it is the price of auditability, and it shifts effort from post-hoc debugging to up-front specification.

Second, the hub is a single point of coordination. One SQLite file suits a single operator and not multi-tenant or geo-replicated use. We chose availability-by-simplicity: restarting the hub drops in-flight supervision, and we schedule that outside working hours rather than pretending it is free.

Third, clean routing makes tight coupling hard. An executor cannot ask the brain a mid-task question. When a task genuinely needs a decision, it must find an authorized path or deliver a failure report and let the next submission carry the decision. We accept the round trip; others may not.

Threats to validity. The data is one operator's workflow, so capability names and durations reflect one person's task mix; machine-hours are wall-clock, not compute; and the evidence rule was introduced mid-deployment, so we report the unflattering aggregate (79% reported at all, 52% left no gap) beside the current-period trend. A harness makes one agent better; a hub makes many composable and auditable, and conflating them puts durable state in the process most likely to die (Appendix D).

9. Conclusion

The interesting problem in long-horizon agents is not making a model plan better; it is making a system remember, route, and prove. PAD is a small, opinionated answer: all durable state in one hub, nine task tools for the brain, routing by declared capability so topology never enters the prompt, and no completion without evidence per criterion. Two months and 938 work tasks across six heterogeneous machines suggest this holds in the tail—the 19% over an hour and the handful over a day—where single-loop context management is most fragile. The remaining costs are real and, in our view, right to pay.

Acknowledgements

We thank the operators of the six nodes for keeping the fleet alive through the incidents catalogued in §5, and the authors of the open harnesses plugged in behind the capability table.

References

Anthropic. Building effective agents. https://www.anthropic.com/engineering/building-effective-agents, 2024.

Bhatt, M., Narajala, V. S., and Habler, I. ETDI: Mitigating tool squatting and rug pull attacks in model context protocol by using OAuth-enhanced tool definitions and policy-

based access control. *arXiv preprint arXiv:2506.01333*, 2025.

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*, 2025.

Cordis Authors. A programming paradigm for spatiotemporal composability. *arXiv preprint arXiv:2608.25512*, 2026.

DeepSeek-AI. DeepSeek Harness: Everything is a plugin. https://github.com/deepseek-ai/deepseek-harness, 2026. Developer preview; built on Cordis.

Ding, T., Nannapaneni, A., Liu, B., and Zhang, L. Always-on agents: A survey of persistent memory, state, and governance in LLM agents. *arXiv preprint arXiv:2606.30306*, 2026.

Lan, X., Chen, Q., Tao, K., Tang, X., Wang, T., Cao, Q., Kong, X., Wen, Z., Zhang, Z., and Zhou, J. SearchSwarm: Towards delegation intelligence in agentic LLMs for long-horizon deep research. *arXiv preprint arXiv:2606.09730*, 2026.

Lumer, E., Sen, S., Paul, K., and Subbiah, V. K. Recursive agent harnesses. *arXiv preprint arXiv:2606.13643*, 2026.

Ma, Z., Huang, H., Zou, S., Wang, Y., Yang, S., Hu, Y., Wei, F., and Chu, X. LongHorizon-Harness: Advancing long-horizon agents for real-world tasks. *arXiv preprint arXiv:2608.01964*, 2026.

Model Context Protocol. Model context protocol specification, 2025-11-25. https://modelcontextprotocol.io/specification/2025-11-25, 2025.

Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. Ray: A distributed framework for emerging AI applications. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Shen, X., Yan, L., Wang, S., Wu, J., Chen, Z., Tang, J., and Ren, J. Goal-aware dynamic multi-agent collaboration for long-horizon tasks. *arXiv preprint arXiv:2610.39193*, 2026.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. Cognitive architectures for language agents. *arXiv preprint arXiv:2309.02427*, 2023.

Temporal Technologies. Temporal: Durable execution platform. https://docs.temporal.io, 2026.

Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J.-R. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023b.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Yin, X., Wang, X., Pan, L., Lin, L., Wan, X., and Wang, W. Y. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.

Zheng, J., Fang, X., Zhang, J., Gui, Z., Chen, H., and Zhang, N. OneDayAgent: Towards a long-horizon harness for autonomous agents. *arXiv preprint arXiv:2608.05013*, 2026.

Zhou, C., Chai, H., Chen, W., Guo, Z., Shan, R., Song, Y., Xu, T., Yang, Y., Yu, A., Zhang, W., et al. Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*, 2026.

A. What We Deliberately Did Not Build

Design principles are legible in what was refused as much as in what was built. Each refusal below was considered and rejected for a stated reason, and each would have been locally convenient.

A planner in the hub. PAD does not decompose goals into steps. Decomposition benefits most from being close to the machine, and it is the part a stronger model does better each year; putting it in the hub would freeze it at today's capability.

A message bus or a vector store. Neither was needed to answer any question the deployment actually asked. Both would have added a second source of truth, and the first rule of the ledger is that there is one.

Mid-task dialogue. An executor cannot ask the brain a question. This is the sharpest restriction and the most expensive to live with; it is also what keeps the ledger a linear record rather than a conversation graph.

Multi-tenancy. One SQLite file, one operator. We chose availability by simplicity and accept the consequences in §8 rather than designing a distributed ledger we did not need.

A model in the routing path. Capability matching is a table lookup. A model guessing which node should get a task would make routing non-deterministic and un-auditable, and would reintroduce exactly the topology knowledge the design removes.

B. Executor Adapters

A capability maps to either a shell command or a thin adapter that renders a prompt template and invokes an agent CLI headlessly. The four adapters used in the measured deployment:

```
executor   invocation (headless, non-interactive)
---------  ------------------------------------------------------------
opencode   opencode run --standalone --auto \
                        --title pad:<id> <prompt>
claude     claude -p --dangerously-skip-permissions <prompt>
codex      codex exec -C <ws> --skip-git-repo-check \
                        --dangerously-bypass-approvals-and-sandbox <prompt>
script     any command that writes result.json
```

Each adapter is one function in the node daemon. Adding a fifth vendor's CLI is a function plus a YAML entry, and it does not touch the hub, the brain, or any other node.

C. Lifecycle Properties that Had to Be Engineered

Four behaviors are not obvious from the architecture but are necessary for it to survive contact with reality. Each came from a real incident in the deployment, and none of them is discoverable by reasoning about the design on paper.

Cancellation must reach the execution plane. Marking a task cancelled in the database while its process keeps running holds a slot hostage: on a single-slot node, one cancelled task blocks every queued task behind it. Executors now poll a lightweight status endpoint and kill the whole process group on cancellation.

Timeout is not cancellation. A soft session timeout may fire while a descendant process is still doing valid work. We therefore treat timeout as a lease event—harvest the result if it lands—and reserve process killing for explicit cancellation only.

Receipt is not persistence. A node that receives HTTP 200 has not necessarily landed its report; in one incident every node reported success while the hub stored empty payloads. Nodes now read back their own report and retry until they observe it, using the idempotency key.

Skeleton drift is invisible without a fingerprint. Documents and prompt templates live on the hub and are copied to nodes, so a fixed rule only reaches nodes that re-pull. We hash the behavioral files into a short fingerprint, have nodes report theirs at registration, and surface mismatches in the situation view.

D. Relation to Harness-Centric Designs

PAD does not replace an agent harness, and we want to be precise about the boundary because the two are often presented as alternatives.

In the measured deployment the per-node executor *is* a harness CLI—three vendors' worth. Everything a harness does to make one agent better at long-horizon work (context management, tool design, planning loops, self-critique) raises the ceiling on every task PAD routes, because that agent is what actually does the work. PAD contributes nothing to that ceiling.

What PAD contributes is composability and auditability across agents: which machine holds which capability, what was asked, what came back, and what evidence accompanied it. These are properties of a fleet, and they do not appear no matter how good a single harness becomes, because a harness cannot see across processes it did not start.

The failure mode of conflating the two is specific: durable task state ends up inside the process most likely to die. A harness that holds the plan in its own context loses the plan when the context is truncated, the process is OOM-killed, or the machine reboots. That is not a bug in the harness; it is the harness doing its job at the wrong layer.

A useful test for which layer a feature belongs to: *if this machine were replaced by an identical one running the same CLI, would the feature still work?* Routing, acceptance gating, evidence coverage and cancellation all survive that test, so they belong in the hub. Context management, tool selection and planning loops do not, because they are properties of a particular run.

E. The Database Schema

Five tables, append-only. The schema is the whole durable state of the system; nothing else on the hub survives a restart.

```
nodes(node_id PK, role, description, capabilities JSON,
      meta JSON, summary, last_seen)

tasks(task_id PK, capability, goal, constraints JSON,
      acceptance JSON, deadline, priority, status,
      node_id, created_by, created_at, updated_at)
  INDEX (capability, status, priority)

reports(report_id PK AUTOINC, task_id, node_id, level,
        summary, payload JSON, client_id, created_at)
  INDEX (task_id)

artifacts(artifact_id PK AUTOINC, task_id, node_id, name,
          bytes, sha256, rows, schema, local_path,
          stored_path, uploaded_at)
  UNIQUE (task_id, name)

task_inputs(input_id PK AUTOINC, task_id, name, mime_type,
            bytes, sha256, stored_path, created_at)
  UNIQUE (task_id, name)
```

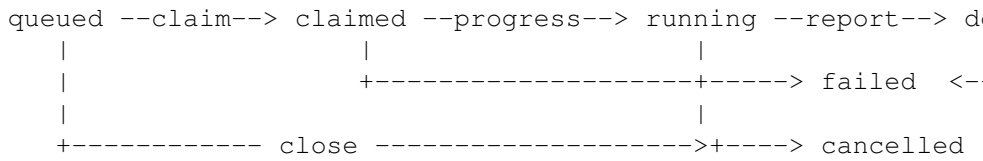
Three details are load-bearing and each was added after an incident.

`created_by` is diagnostic, never routing. The column records which client submitted the task (`chatgpt`, `human`, or a timer name). No query in the hub branches on it; it exists so that the deployment can be split by brain mode after the fact, which is how §4 got its numbers. Adding a fourth client requires no schema change and no hub change.

`client_id` on reports makes retries safe. A node that loses a response retries with the same client-supplied key; the insert is a no-op on conflict. This is the mechanism behind "receipt is not persistence" (Appendix C): a node reads back what the hub stored and retries until it observes its own report.

The queue index is `(capability, status, priority)`. Claiming is one indexed query ordering by priority then age, which means queueing order is reproducible and inspectable rather than emergent. A backlog is a `SELECT COUNT(*)` on that index.

F. The Task Lifecycle



A claim is atomic: `BEGIN IMMEDIATE`, select one row by capability and priority, update status, commit. The hub uses SQLite's write lock rather than an external lock service, which is sufficient because claims are short and the fleet is small. A node that dies after claiming leaves the row in `claimed`; the lease expires and the task returns to `queued` rather than being lost.

Cancellation is a second path into the terminal set, and it is the only one that requires the executor to cooperate (Appendix C).

G. The Nine Tools, in Detail

The tool surface is the authority surface. Each tool below is chosen against the question "does the brain need this, or does the executor already have it?"; every tool that acts on a machine was rejected and lives in the executor instead.

Tool	Contract
`get_digest`	nodes, capabilities with descriptions, active tasks, per-node recent task ids, stale nodes, skeleton mismatches, done-today count
`submit_task`	capability, goal, constraints, acceptance, deadline, priority, input files; returns task id
`get_task`	status, full report stream with payloads, artifacts with schema and row counts, input file list
`get_artifact`	paged read of artifact bytes by offset, so a large file never enters the context at once
`get_input_file`	read a file dispatched with the task
`close_task`	cancel; propagates to the executor's process group
`probe_file_upload`	verify the attachment path end to end before committing to a long task
`get_artifact_delivery`	signed external link (optional)
`get_artifact_observe`	sampled frames from a media artifact (optional)

Two of the nine exist because of a failure mode rather than a capability. `probe_file_upload` exists because attachments silently failed for a week before anyone noticed; it runs the same path a real submission would, and returns the first place it breaks. `get_artifact` is paged because a 200 MB artifact returned in one call would consume the context the design exists to protect.

What is deliberately absent is as important: no shell, no filesystem write, no arbitrary HTTP fetch, no machine name in any argument, and no way to run a command. The only network egress the hub performs on behalf of an external party is the fetch-by-URL path used to pull user attachments, which refuses private and loopback resolution, re-validates every redirect hop, and aborts on size limits.

H. The Submission Contract

A task is four fields and two policies. Writing them well is the operator's main skill in this system, and §8 treats that as a cost rather than a feature.

Goal. One paragraph stating the outcome, not the steps. A goal that names a method removes the executor's ability to find a better one.

Constraints. What must not happen and what must stay true: no publishing without review, do not modify the source tree, stay under a size limit, do not touch a named account.

Acceptance criteria. A list of checks that a machine can decide. "Report covers every announcement from this week" is decidable; "report is good" is not. The hub's coverage record is only as meaningful as this list, which is why vague criteria produce vague answers and we say so in §8.

Deadline and priority. A deadline is a lease, not a kill signal (Appendix C).

The two policies are: failed tasks must still deliver a structured report, and raw volume stays on the node while the hub keeps a pointer with row counts and schema.

I. Reproducing the Measurements

```
python3 tools/collect_stats.py \
        --out paper/data/tasks.json
python3 tools/make_paper_data.py \
        paper/data/tasks.json paper/data/aggregates.json
python3 tools/classify_failures.py
```

The first script reads the situation endpoint with a large recent-task window and fetches each task record concurrently. The second reduces the raw dump to the aggregates in Tables 3–4, because the raw records contain real goals and task identifiers and are operator data. The third applies the fixed first-match rule behind Table 4, so the same input always yields the same table.